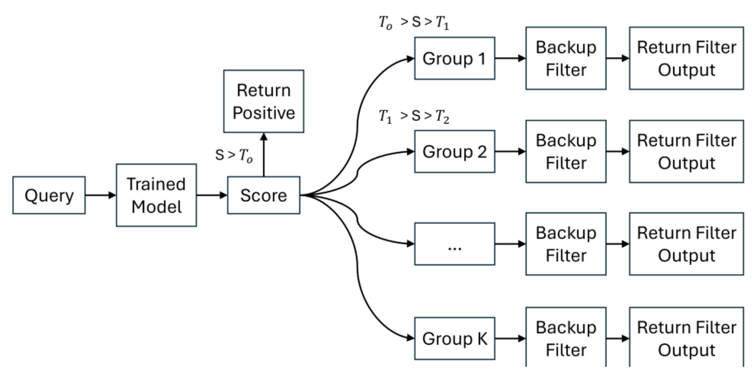
Evaluating Filters Augmented with Adaptation vs ML Models

Diandre Sabale, Prashant Pandey

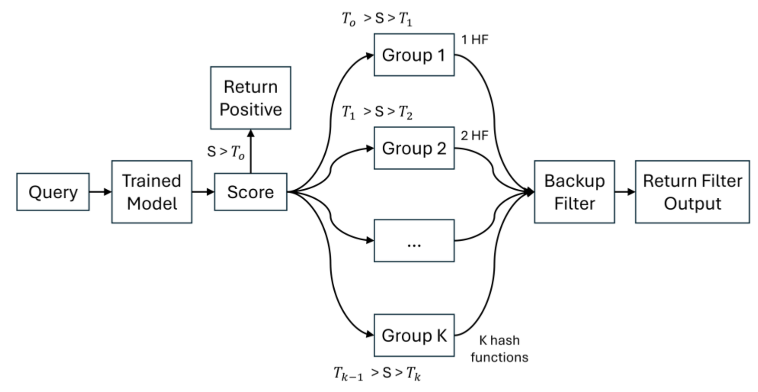
diandremiguels.github.io

Learned Filters

Use machine learning models to incorporate information about key features into query decisions

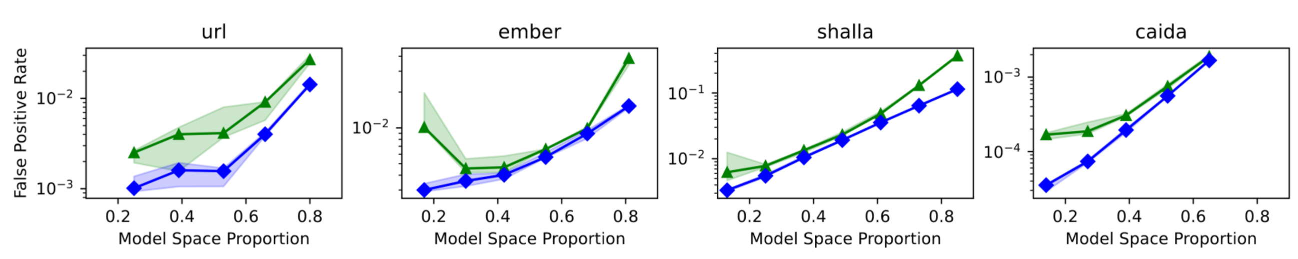


The **PLBF** [1] is the state-of-the-art learned filter, assigning keys to different backup filters based on model scores



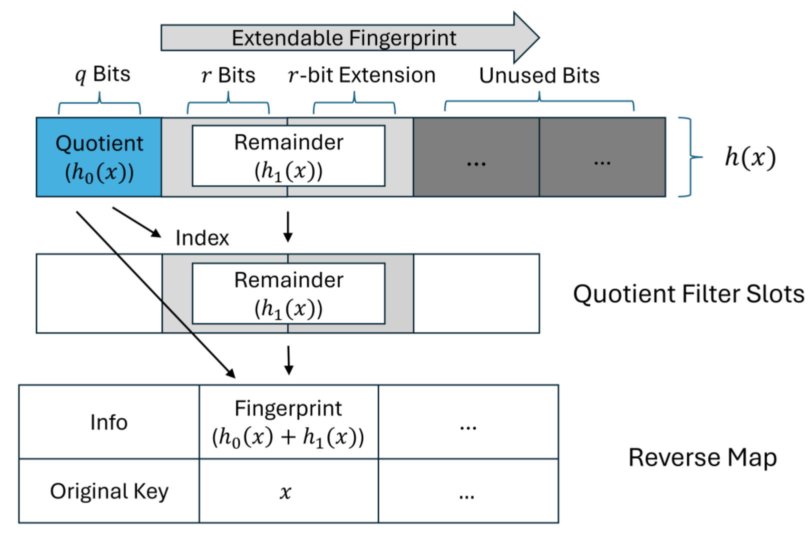
The **Ada-BF** [2] is the next best learned filter, assigning differing numbers of hash functions to keys based on model scores

Need to strike a balance between model and backup filter size



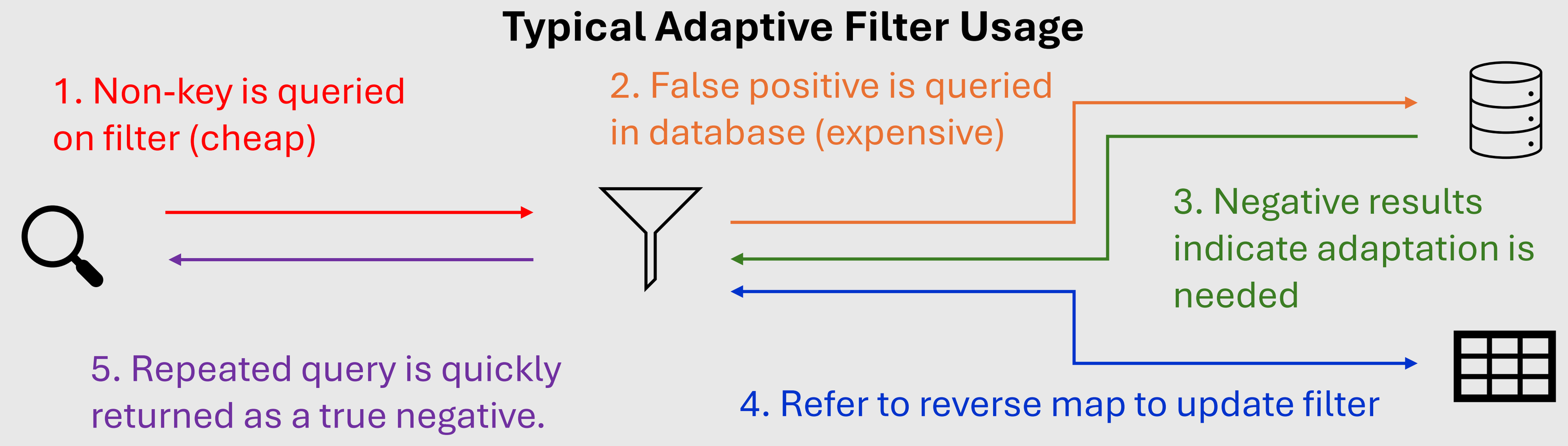
Adaptive Filters

Change structure in response to false positives to reduce future mistakes



The **Adaptive Quotient Filter** [3] is the state-of-the-art adaptive filter, making use of variable-length fingerprints to avoid hash collisions

Typical Adaptive Filter Usage



Problem Setting

- **Learned** and **adaptive** filters represent major paradigm shifts over traditional filters
- We identify critical gaps in evaluation across and within these paradigms
  - **Query distribution** assumptions
  - Missing **model cost** analysis
  - Niche use cases and **dataset selection**
  - **Theoretical** versus **empirical** guarantees
- Notably, there is **no direct comparison** of learned and adaptive filters

Contributions

- ***Compare theoretical guarantees*** of each filter type, with experimental validation
- ***Analyze effects of construction decisions*** on learned filter accuracy
- ***Conduct extensive performance evaluation*** with real-world workloads and datasets
- ***Provide evidence-based recommendations*** for use cases for each filter

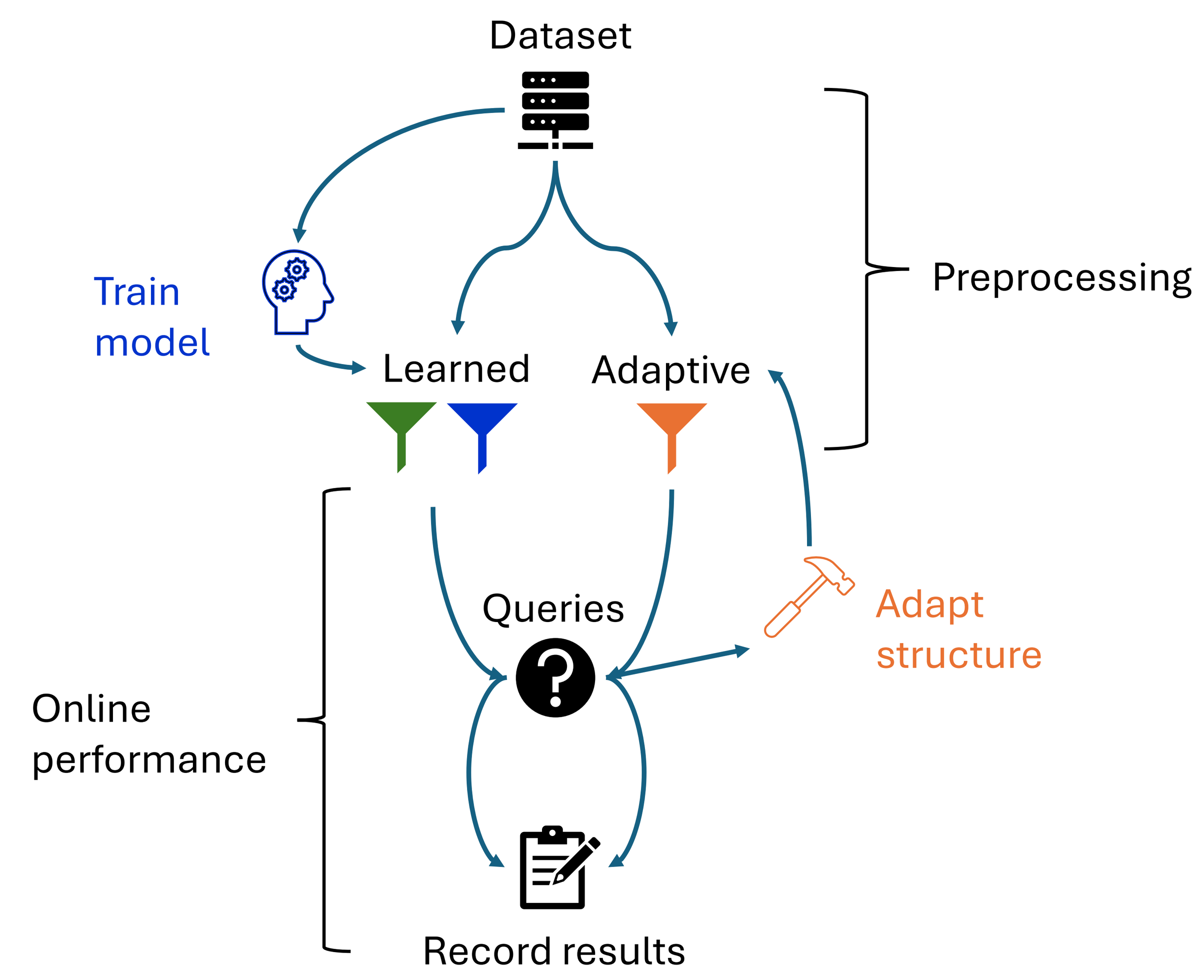
Experimental Setup

Real-world datasets
- URL – (non) malicious URLs
- Ember – (malicious) software metadata
- Shalla – (non) malicious URLs
- Caida – network traces

Varying query distributions
- Non-skewed (one-pass, uniform)
- Skewed (Zipfian, adversarial)
- Dynamic (with and without model retraining)

Direct performance comparison
- Use both learned filters (with Random Forest Classifiers) and the AdaptiveQF
- Run 10 million queries and report effect of overall filter size on FPR on each dataset and query distribution
- Also record construction and query times for each filter type

Evaluation Workflow



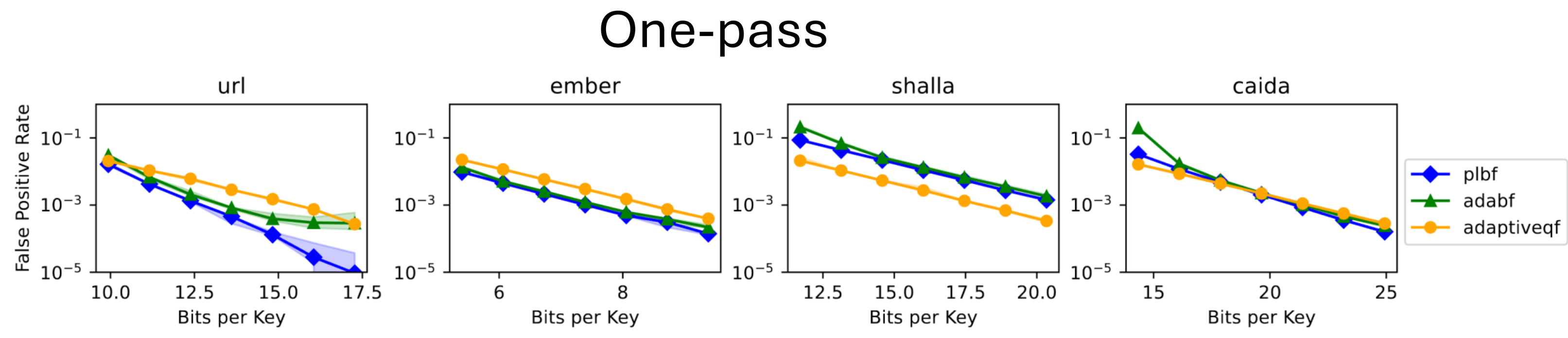
1. Sato, A., & Matsui, Y. (2023). Fast partitioned learned bloom filter. *Advances in Neural Information Processing Systems*, *36*, 39119-39146.
2. Dai, Z., & Shrivastava, A. (2020). Adaptive learned bloom filter (ada-bf): Efficient utilization of the classifier with application to real-time information filtering on the web. *Advances in neural information processing systems*, *33*, 11700-11710.
3. Wen, R., McCoy, H., Tench, D., Tagliavini, G., Bender, M. A., Conway, A., ... & Pandey, P. (2024). Adaptive Quotient Filters. *Proceedings of the ACM on Management of Data*, *2*(4), 1-28.

Results

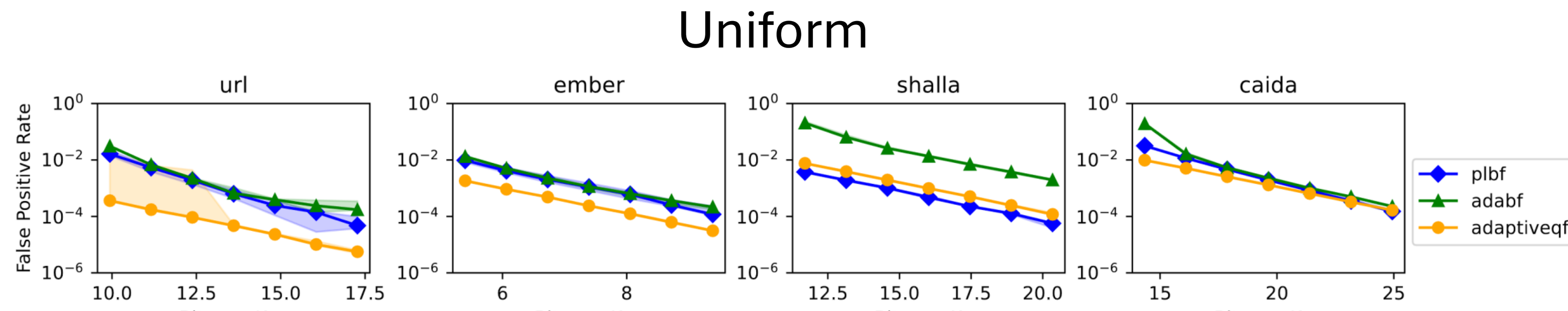
Learned filters are most suitable for stable and highly predictable workloads where space efficiency is paramount, while adaptive filters are a more generalizable option which can handle dynamic environments dependably.

On non-skewed workloads, both filter paradigms perform similarly

One-pass

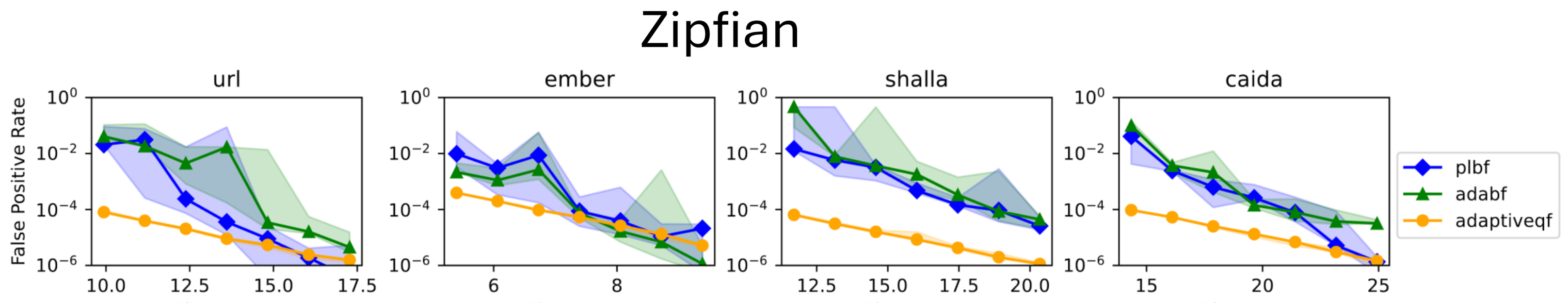


Uniform

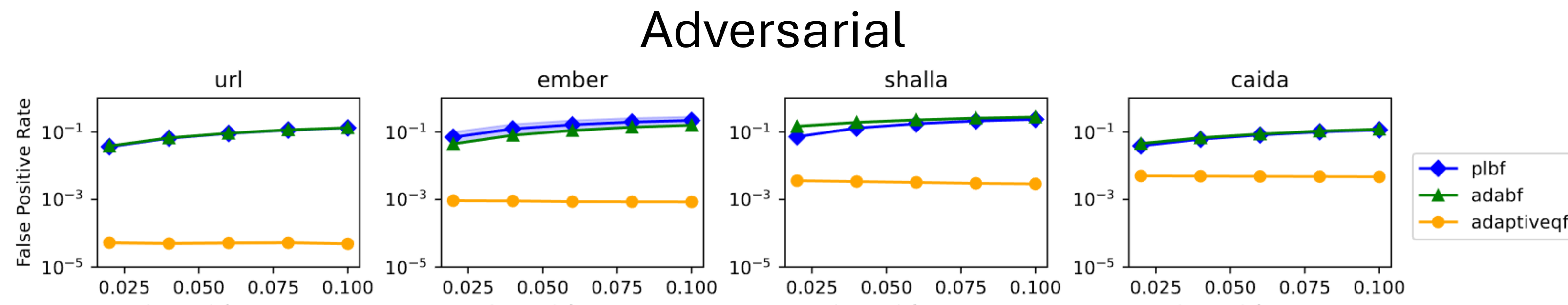


On skewed workloads, learned filters have variable and higher FPRs

Zipfian

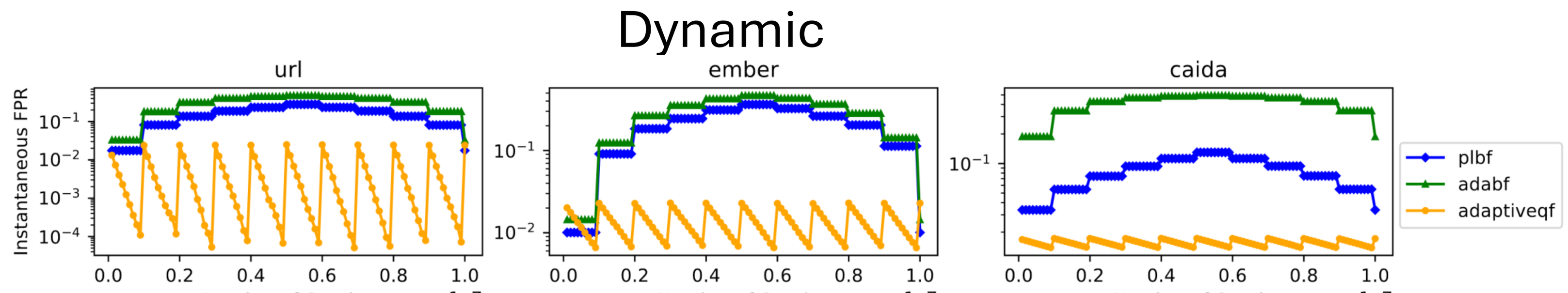


Adversarial

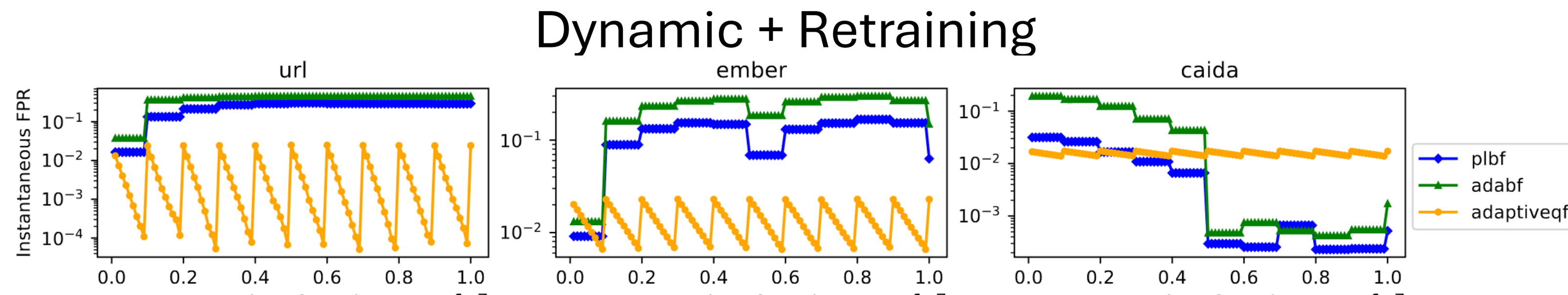


On dynamic workloads, adaptive filters have robust lower FPRs

Dynamic

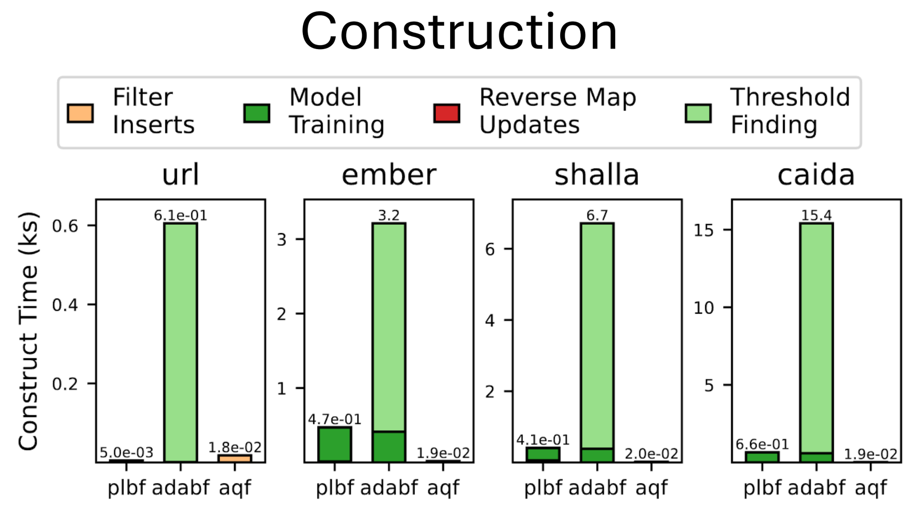


Dynamic + Retraining



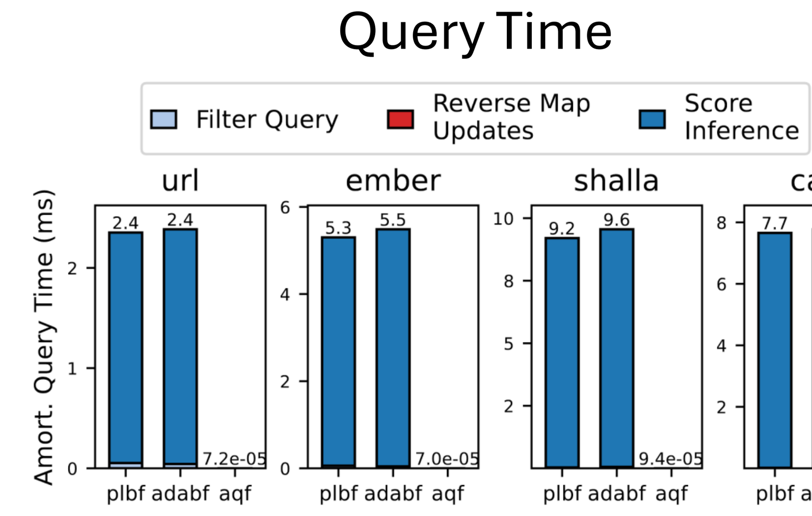
The cost of learned model operations is a significant bottleneck on performance (up to 10^4x worse query latency)

Construction



Model training takes longer than AdaptiveQF construction!

Query Time



Model score inference takes longer than AdaptiveQF queries!